

UT 07 – Vistas y otros objetos de la Base de Datos.



Autor: Carlos Moreno Martínez.
cmorenomartinez@educa.madrid.org



UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.1.- Introducción. Definición.

- Muchas veces algunas consultas requieren un cierto tiempo en realizar debido a continuos errores y refinados. Esto implica dos cosas:
 - **Si estas consultas se necesita ejecutarlas frecuentemente implica una perdida de tiempo el hecho de redactarlas cada vez que se necesite.**
 - **Un usuario poco experto, puede ser incapaz de desarrollarla.** Sin embargo, necesita el resultado de la consulta para su trabajo. Para este usuario le resultaría mas fácil que los datos se le mostraran como una tabla simple en la que puede redactar consultas sencillas.
 - Se puede dar permisos de uso sobre una vista que **visualiza parcialmente la información de una varias tablas, ocultando información sensibles.**
- Ejemplo.** Se puede dar la información sobre tarjetas bancarias sin mostrar el PIN que será comprobado mediante otros métodos seguros.

Definición de Vista.

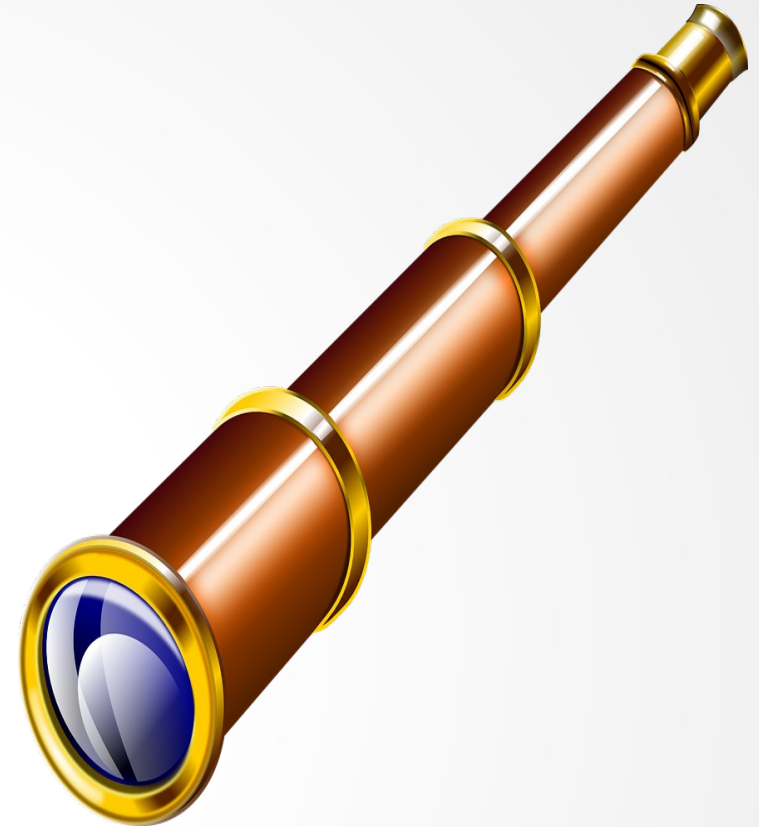
- **Una vista es una tabla virtual que almacena una consulta. Los datos accesibles a través de la vista están almacenados en la base de datos no en la vista.**
- **La SGBD guarda la definición de la vista. Cuando se hace una consulta a la vista, el SGBD recupera la información de la base de datos.**
- **Las vistas tienen la misma estructura que una tabla con sus respectivas filas y columnas, pudiéndose tratar por tanto de forma semejante a una tabla.**

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.1.- Introducción. Casos de utilización.

- Además de esto, las vistas tienen otras ventajas:
 - **Simplificar la administración de los permisos de usuario.** Se pueden dar al usuario permisos para que solamente pueda acceder a los datos a través de vistas, en lugar de concederle permisos para acceder a ciertos campos, así se protegen las tablas base de cambios en su estructura.
 - **Mejorar el rendimiento.** Se puede evitar escribir instrucciones repetidamente almacenando en una vista el resultado de una consulta compleja que incluya información de varias tablas. Esta información puede almacenarse en cache y ahorrar tiempo de respuesta y de ejecución.
 - **Si se suprime una vista de la base de datos, las tablas que consulta no son afectadas.** Sin embargo, si se suprime una tabla, la vista asociada se invalida.



UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.2.- Privilegios necesarios para crear una vista.



- Por defecto, un usuario no puede gestionar vistas. Tiene que tener el privilegio **CREATE VIEW**. Este privilegio se le concede mediante un usuario administrativo como por ejemplo **SYSTEM** mediante el comando:

GRANT CREATE VIEW TO [USUARIO];

- Por ejemplo, para nuestro caso estamos usando el perfil de ejemplo SCOTT que no tiene el privilegio de creación de vistas (tan solo tiene como privilegios CONNECT y RESOURCE).

```
GRANT CREATE VIEW TO SCOTT;
```

```
Grant succeeded.
```

- Para concederle el privilegio de creación de vistas ejecutaremos como SYSTEM.

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.3.- Vistas en el diccionario de datos (I). Tabla USER_VIEWS.

- Para comprobar que vistas hay en el Sistema y que cual es su cometido, en el diccionario de datos existe una serie de tablas vistas que nos permiten obtener información sobre ellas.
- La primera es **USER_VIEWS**. Esta vista permite a un usuario comprobar que vistas tiene disponible en su perfil. La descripción es la siguiente:

```
DESC USER_VIEWS;
```

Name	Null?	Type
-----	-----	-----
VIEW_NAME	NOT NULL	VARCHAR2 (30)
EXT_LENGTH		NUMBER
TEXT		LONG
TYPE_TEXT_LENGTH		NUMBER
TYPE_TEXT		VARCHAR2 (4000)
OID_TEXT_LENGTH		NUMBER
OID_TEXT		VARCHAR2 (4000)
VIEW_TYPE_OWNER		VARCHAR2 (30)
VIEW_TYPE		VARCHAR2 (30)
SUPERVIEW_NAME		VARCHAR2 (30)
EDITIONING_VIEW		VARCHAR2 (1)
READ_ONLY		VARCHAR2 (1)

- Entre los campos importantes podemos destacar dos:
 - **VIEW_NAME**. Indica el nombre de la vista creada. En este campo aparecerán los vistas creadas en el perfil en curso.
 - **TEXT**. Indica el código de la vista. esto es útil para conocer el resultado de las consultas SQL.

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.3.- Vistas en el diccionario de datos (II). Tabla ALL_VIEWS.

- Otra vista importante es **ALL_VIEWS**. Esta vista del diccionario permite ver las vistas presentes en el Sistema. La descripción es la siguiente:

```
DESC ALL_VIEWS;
```

Name	Null?	Type
OWNER	NOT NULL	VARCHAR2 (30)
VIEW_NAME	NOT NULL	VARCHAR2 (30)
TEXT_LENGTH		NUMBER
TEXT		LONG
TYPE_TEXT_LENGTH		NUMBER
TYPE_TEXT		VARCHAR2 (4000)
OID_TEXT_LENGTH		NUMBER
OID_TEXT		VARCHAR2 (4000)
IEW_TYPE_OWNER		VARCHAR2 (30)
VIEW_TYPE		VARCHAR2 (30)
SUPERVIEW_NAME		VARCHAR2 (30)
EDITIONING_VIEW		VARCHAR2 (1)
READ_ONLY		VARCHAR2 (1)

- La vista es muy semejante al anterior salvo el primer campo **OWNER** que representa el propietario de la vista.

Para poder usar una vista, se debe de tener privilegios sobre dicha vista.

UT 07 – Vistas y otros objetos de la Base de Datos.

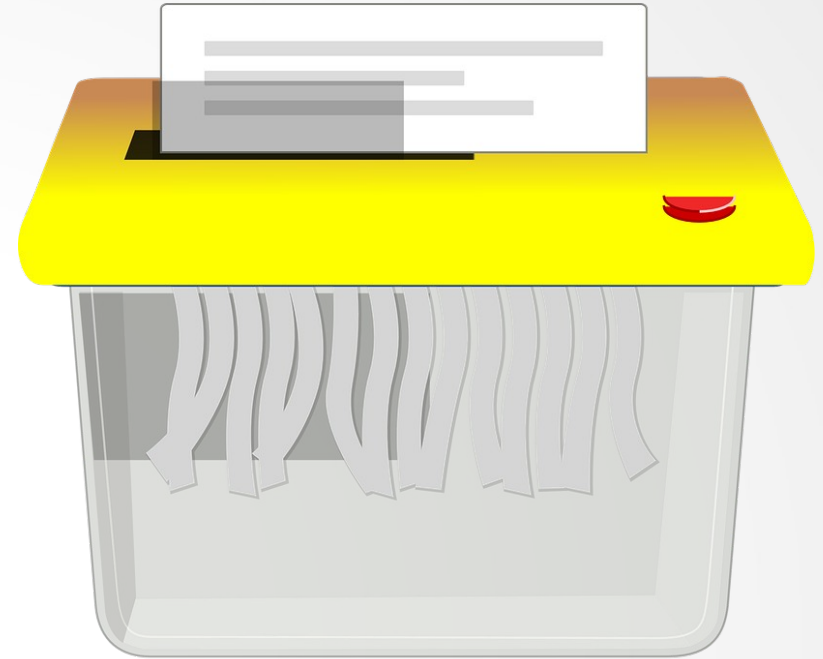
1.- Vistas.

1.4.- Borrado de una vista presente en el Sistema.

- Para eliminar una vista se usa el comando DROP VIEW. La sintaxis es la siguiente:

DROP VIEW [nombre];

- A partir de ese momento, **la vista desaparece del Sistema.**
- **El uso del comando DROP VIEW es irrevocable;** una vez realizado no es posible la recuperación de la vista borrada.



UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.5.- Creación de una Vista. Formato de definición.

- La sintaxis básica para crear una vista es la siguiente:

```
CREATE [OR REPLACE] VIEW NombreVista  
[(columna [,columna])]  
AS  
consulta;
```

- Si no se incluyen columnas se toman los nombres de los campos de la consulta y sus alias si están definidos (siempre que no se incluyan espacios en blanco).
- Opcionalmente puede ir acompañado de **OR REPLACE**. En este caso, si la vista no existe se crea y en caso contrario se actualiza.

```
CREATE OR REPLACE VIEW NOMINA AS  
SELECT  
    EMPNO,  
    ENAME,  
    JOB,  
    SAL,  
    COALESCE(SAL + COMM, SAL) TOTAL  
FROM EMP;
```

View NOMIMA creado.

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.5.- Creación de una Vista. Forma alternativa de definir una vista.

- Como puede verse en el ejemplo, **si no se incluyen los nombres de columnas con lo cual se toman de los nombres de la tablas y del alias de la columna calculada.**
- Sin embargo, podríamos usar el formato estándar de la definición.

```
CREATE OR REPLACE VIEW NOMINA
(EMPID, NOMBRE, PUESTO, TOTAL)
AS
SELECT
    EMPNO,
    ENAME,
    JOB,
    COALESCE(SAL + COMM, SAL)
FROM EMP;
```

View NOMINA creado.

```
-- MOSTRAR EL ID DE EMPLEADO, NOMBRE Y TOTAL DE LOS EMPLEADOS QUE
-- GANEN MAS DE 2,750.
```

```
SELECT EMPID, NOMBRE, TOTAL
FROM NOMINA
WHERE TOTAL > 2750
ORDER BY TOTAL;
```

EMPID	NOMBRE	TOTAL
7698	BLAKE	2850
7566	JONES	2975
7902	FORD	3000
7788	SCOTT	3000
7839	KING	5000

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.6.- Consulta de los datos de una vista.

- Una vez creada, se puede consultar como si fuera una tabla mas del Sistema.

```
-- MOSTRAR LA NOMINA DE LOS EMPLEADOS QUE SEAN CLERK.
```

```
SELECT * FROM NOMINA  
WHERE JOB = 'CLERK';
```

EMPNO	ENAME	JOB	SAL	TOTAL
7369	SMITH	CLERK	800	800
7876	ADAMS	CLERK	1100	1100
7900	JAMES	CLERK	950	950
7934	MILLER	CLERK	1300	1300

```
-- MOSTRAR EL ENOMBRE, OFICIO Y TOTAL DE LOS EMPLEADOS QUE COBREN ENTRE  
-- 2,500 Y 3,000.
```

```
SELECT ENAME, JOB, TOTAL FROM NOMINA  
WHERE TOTAL BETWEEN 2500 AND 3000  
ORDER BY TOTAL;
```

ENAME	JOB	TOTAL
MARTIN	SALESMAN	2650
BLAKE	MANAGER	2850
JONES	MANAGER	2975
FORD	ANALYST	3000
SCOTT	ANALYST	3000

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.7.- Uso de vistas en consultas elaboradas.

- Como se decía anteriormente, las vistas sirven para ahorrar tiempo en consultas un tanto laboriosas.

```
CREATE OR REPLACE VIEW NOMINA AS
SELECT
    E.EMPNO,
    E.ENAME,
    E.SAL,
    E.COMM,
    COALESCE(SAL+COMM, SAL) TOTAL,
    E.DEPTNO,
    D.DNAME,
    D.LOC,
    S.GRADE
FROM EMP E JOIN DEPT D ON (E.DEPTNO = D.DEPTNO)
JOIN SALGRADE S ON (E.SAL BETWEEN S.LOSAL AND S.HISAL);
```

View NOMIMA creado.

```
-- MOSTRAR EL NUMERO DE EMPLEADO, NOMBRE, TOTAL Y CATEGORIA SALARIAL DE TODOS
-- LOS EMPLEADOS DEL DPTO 10.
SELECT EMPNO, ENAME, TOTAL, GRADE FROM NOMINA
WHERE DEPTNO = 10;
```

EMPNO	ENAME	TOTAL	GRADE
7839	KING	5000	5
7782	CLARK	2450	4
7934	MILLER	1300	2

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.8.- Sentencias DML (INSERT, UPDATE o DELETE) dentro de Vistas. Excepciones.



- No es posible realizar operaciones DML en los casos en los que la consulta asociada a la vista tenga las siguientes características:
 - Se realizan agrupamientos (cláusula **GROUP BY**).
 - Contiene funciones de agrupación (**SUM, COUNT, AVG, etc**).
 - Se usa la cláusula **DISTINCT**.
 - Se usa la pseudocolumna **ROWNUM**.
 - Se utilizan expresiones o funciones.
 - Contiene **JOINS**.
 - Contiene subconsultas.
- Todas las excepciones guardan su lógica: **los datos han sido obtenidos realizando cálculos y contabilizaciones del contenidos de las tablas y se crearía incongruencias en los datos.**

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.8.- Sentencia DML (INSERT, UDATE o DELETE) dentro de Vistas. Ejemplos.

```
CREATE OR REPLACE VIEW DEPT_NUMEMP AS
SELECT
    D.DNAME,
    COUNT(E.ENAME) NUMEMP
FROM EMP E RIGHT JOIN DEPT D ON (E.DEPTNO = D.DEPTNO)
GROUP BY D.DNAME;
```

View DEPT_NUMEMP creado.

```
SELECT * FROM DEPT_NUMEMP;
```

DNAME	NUMEMP
ACCOUNTING	3
OPERATIONS	0
RESEARCH	5
SALES	6

```
-- CAMBIAR EL NOMBRE DE SALES A MARKETING.
UPDATE DEPT_NUMEMP
SET DNAME = 'MARKETING'
WHERE DNAME = 'SALES';
```

Error SQL: ORA-01732: data manipulation operation not legal on this view
01732. 00000 - "data manipulation operation not legal on this view"

```
-- AUMENTAR EN UN EMPLEADO LA PLANTILLA DE SALES.
UPDATE DEPT_NUMEMP
SET NUMEMP = NUMEMP + 1
WHERE DNAME = 'SALES';
```

Error SQL: ORA-01732: data manipulation operation not legal on this view
01732. 00000 - "data manipulation operation not legal on this view"

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.9.- Sentencia DML (INSERT, UPDATE o DELETE) dentro de Vistas. Casos posibles.

- Hay casos en los cuales **es posible realizar operaciones de manipulación de datos dentro de una vista.**
- Se podrá cuando se cumplan las siguientes condiciones:
 - **Los cambios no deben afectar a más de una tabla consultada.** Pueden insertarse, modificarse y eliminarse datos de una vista que combina varias tablas pero **la modificación o eliminación solamente debe afectar a una sola tabla.**
Para poder realizar este tipos de operaciones existen los denominados **Triggers**, que se verán en un posterior documento.
 - **Los cambios pueden afectar a campos que no están en la vista.** Debe tenerse en cuenta que cuando no se insertan datos a través de una vista, **el resto de los campos que no están referenciados en ella quedan a NULL, salvo los campos que tengan un valor DEFAULT definido en la tabla al crearla.**
Eso puede dar problemas tales como violaciones de integridad referencial, restricciones NOT NULL, etc.
 - **No es posible realizar inserciones o modificaciones sobre campos que se han obtenidos mediante expresiones o funciones.** Ello da como resultado incongruencias.

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.10.- Vistas de solo lectura. La cláusula READ ONLY.

- **La vista se bloquea en modo lectura.** De esta forma, cuando se intenta realizar una operación DML sobre dicha vista se devuelve un mensaje de error.
- La sintaxis es la siguiente:

```
CREATE VIEW nombre  
AS consulta  
WITH READ ONLY;
```

- Si a WITH READ ONLY se añade **CONSTRAINT [nombre]**, al intenta realizar operaciones DML en filas, se mostrará un error, junto con el nombre de restricción si se ha especificado.

```
CREATE OR REPLACE VIEW EMP30 AS  
SELECT EMPNO, ENAME, SAL  
FROM EMP  
WHERE DEPTNO = 30  
WITH READ ONLY;
```

View EMP30 creado.

```
DELETE FROM EMP30  
WHERE SAL < 5000;
```

```
Error SQL: ORA-42399: cannot perform a DML operation on a read-only view  
42399.0000 - "cannot perform a DML operation on a read-only view"
```

UT 07 – Vistas y otros objetos de la Base de Datos.

1.- Vistas.

1.11.- Vistas de solo lectura. La cláusula WITH CHECK OPTION.

- La cláusula WITH CHECK OPTION permite implementar un cierto nivel de integridad referencial. La sintaxis de la sentencia es la siguiente:

```
CREATE VIEW nombre  
AS consulta  
WITH CHECK OPTION;
```

- Si se intenta realizar operaciones DML en determinadas filas que desaparecerían de la vista se mostrará un error.

```
CREATE OR REPLACE VIEW EMP30 AS  
SELECT EMPNO, ENAME, SAL, DEPTNO  
FROM EMP  
WHERE DEPTNO = 30  
WITH CHECK OPTION;
```

View EMP30 creado.

```
UPDATE EMP30  
SET DEPTNO = 20  
WHERE DEPTNO = 30;
```

ORA-01402: view WITH CHECK OPTION where-clause violation

```
UPDATE EMP30  
SET SAL = SAL + 1000  
WHERE SAL < 1000;
```

1 fila actualizadas.

- **Objeto de base de datos creado por el usuario que pueden compartir varios usuarios para generar enteros.**
- Un uso habitual para las secuencias es la **obtención de forma automática de un valor para una clave primaria, que debe ser único para cada fila.**

La secuencia se genera y se incrementa (o disminuye) en una rutina interna de Oracle.

- Reduce la cantidad de código de aplicación necesario para escribir una rutina de generación de secuencias.
- La eficacia de acceso a los valores de secuencia es alta cuando se almacenan en caché en la memoria.
- Los números de secuencias se almacenan y se generan independientemente de las tablas. Por lo tanto, se puede utilizar la misma secuencia para varias tablas.



- La sintaxis para la creación de una secuencia es la siguiente:

```
CREATE SEQUENCE nombre  
  [INCREMENT BY n]  
  [START WITH n]  
  [{MINVALUE n | NOMINVALUE}]  
  [{MAXVALUE n | NOMAXVALUE}]  
  [{CYCLE | NOCYCLE}]  
  [{CACHE n | NOCACHE}];
```

- Los parámetros de la secuencia son los siguientes:
 - **INCREMENT BY n.** Intervalo entre números de secuencia, donde n es un entero. Si se omite esta cláusula, la secuencia se incrementa en uno.
 - **START WITH n.** Primer número de secuencia que se generará. Si se omite esta cláusula, la secuencia empieza en uno.
 - **MINVALUE n.** Valor mínimo de secuencia. Si se indica NOMINVALUE especifica un valor mínimo de uno para una secuencia ascendente y de -1026 para una descendente. Ésta es la opción por defecto.
 - **MAXVALUE n.** Valor máximo de la secuencia. Si se indica el parámetro NOMAXVALUE especifica un valor máximo de 1027 para una secuencia ascendente y de menos uno para una descendente. Ésta es la opción por defecto.
 - **CYCLE | NOCYCLE.** Especifica si la secuencia continúa generando valores después de alcanzar sus valores máximo o mínimo. NOCYCLE es la opción por defecto.
 - **CACHE n | NOCACHE.** Especifica cuántos valores preasigna y mantiene en memoria Oracle Server. Por defecto, Oracle Server almacena en caché 20 valores.

- Una vez creada la secuencia, podemos hacer referencia a los valores de secuencia mediante las pseudocolumnas **NEXTVAL** y **CURRVAL**.
- **NEXTVAL devuelve el siguiente valor de secuencia disponible.**
 - Devuelve un valor único cada vez que se hace referencia al mismo, incluso para usuarios distintos.
 - Al hacer referencia a **secuencia.NEXTVAL**, se genera un nuevo número de secuencia.
 - El número de secuencia actual se coloca en **secuencia.CURRVAL**.
- **CURRVAL hace referencia a el último número de secuencia que se ha generado.**
 - Antes de que se pueda hacer referencia a CURRVAL, se debe utilizar NEXTVAL para generar un número de secuencia en la sesión del usuario actual.

UT 07 – Vistas y otros objetos de la Base de Datos.

2.- Secuencias.

2.4.- Uso de una secuencia. Ejemplo (I).

Supongamos que se desea gestionar una tabla de empleados. Para crear los valores de la clave primaria (empleadoid), se crea la secuencia **EMPLEADOID_SEQ** de la forma siguiente:

```
CREATE SEQUENCE EMPLEADOID_SEQ  
START WITH 1500  
INCREMENT BY 1  
CACHE 10;  
  
Sequence EMPLEADOID_SEQ creado.
```

También se crea la tabla **EMPLEADOS** de la forma siguiente:

```
CREATE TABLE EMPLEADOS (  
    EMPLEADOID NUMBER(6),  
    NOMBRE VARCHAR2(50),  
    FECHNACIMIENTO DATE,  
    CONSTRAINT EMPLEADOS_EMPLEADOID_PK PRIMARY KEY (EMPLEADOID)  
);  
  
Table EMPLEADOS creado.
```

A continuación veremos como se debe usar la secuencia para generar valores para EMPLEADOID.

UT 07 – Vistas y otros objetos de la Base de Datos.

2.- Secuencias.

2.4.- Uso de una secuencia. Ejemplo (II).

Insertamos dos registros en la tabla EMPLEADOS:

```
INSERT INTO EMPLEADOS
VALUES
(EMPLEADOID_SEQ.NEXTVAL, 'PEDRO LOPEZ', TO_DATE('10/05/1978', 'DD/MM/YYYY'));
```

1 fila insertadas.

```
INSERT INTO EMPLEADOS
VALUES
(EMPLEADOID_SEQ.NEXTVAL, 'ANA JIMENEZ', TO_DATE('30/06/1981', 'DD/MM/YYYY'));
```

1 fila insertadas.

EMPLEADOID	NOMBRE	FECHNACIMIENTO
1500	PEDRO LOPEZ	10/05/78
1501	ANA JIMENEZ	30/06/81

```
SELECT EMPLEADOID_SEQ.CURRVAL FROM DUAL;
```

CURRVAL

1501

UT 07 – Vistas y otros objetos de la Base de Datos.

2.- Secuencias.

2.5.- Perdidas de números de secuencia.

- Hay situaciones en se pueden producir intervalos en los valores de secuencia cuando:
 - Se produce un Rollback.
 - Se produce un error en el sistema.
 - Se utiliza una secuencia en mas de una tabla.
- **Si hace Rollback de una sentencia que contiene una sentencia, se perderá el número.**

```
INSERT INTO EMPLEADOS  
VALUES (seq1.NEXTVAL, 'ENESTO MARTINEZ', TO_DATE('20/05/1991', 'DD/MM/YYYY'));
```

```
1 row created.
```

```
ROLLBACK;
```

```
Rollback complete.
```

```
INSERT INTO EMPLEADOS  
VALUES (seq1.NEXTVAL, 'CARMEN TORRES', TO_DATE('17/11/1987', 'DD/MM/YYYY'));
```

```
1 row created.
```

EMPLEADOID	NOMBRE	FECHNACIMIENTO
1500	PEDRO LOPEZ	10/05/78
1501	ANA JIMENEZ	30/06/81
1503	CARMEN TORRES	17/11/87



Para modificar una secuencia se utiliza el comando **ALTER SEQUENCE**. Su sintaxis es la siguiente:

```
ALTER SEQUENCE sequence  
[INCREMENT BY n]  
[{MAXVALUE n | NOMAXVALUE}]  
[{MINVALUE n | NOMINVALUE}]  
[{CYCLE | NOCYCLE}]  
[{CACHE n | NOCACHE}];
```

Para poder realizar modificaciones en una secuencia es necesario cumplir los siguientes requisitos:

- **Debe ser el propietario de la secuencia o tener el privilegio ALTER para modificarla.**
- Sólo los números de secuencia futuros se ven afectados por la modificación de la secuencia. Es decir, **no es posible modificar números ya asignados.**
- **No es posible modificar la opción START WITH.** Se debe borrar la secuencia y volver a crearla para reiniciarla en un número diferente.
- Se llevan a cabo algunas validaciones. **Por ejemplo, no se puede imponer un nuevo valor MAXVALUE que sea menor que el número actual de secuencia.**

Borrado de una secuencia.

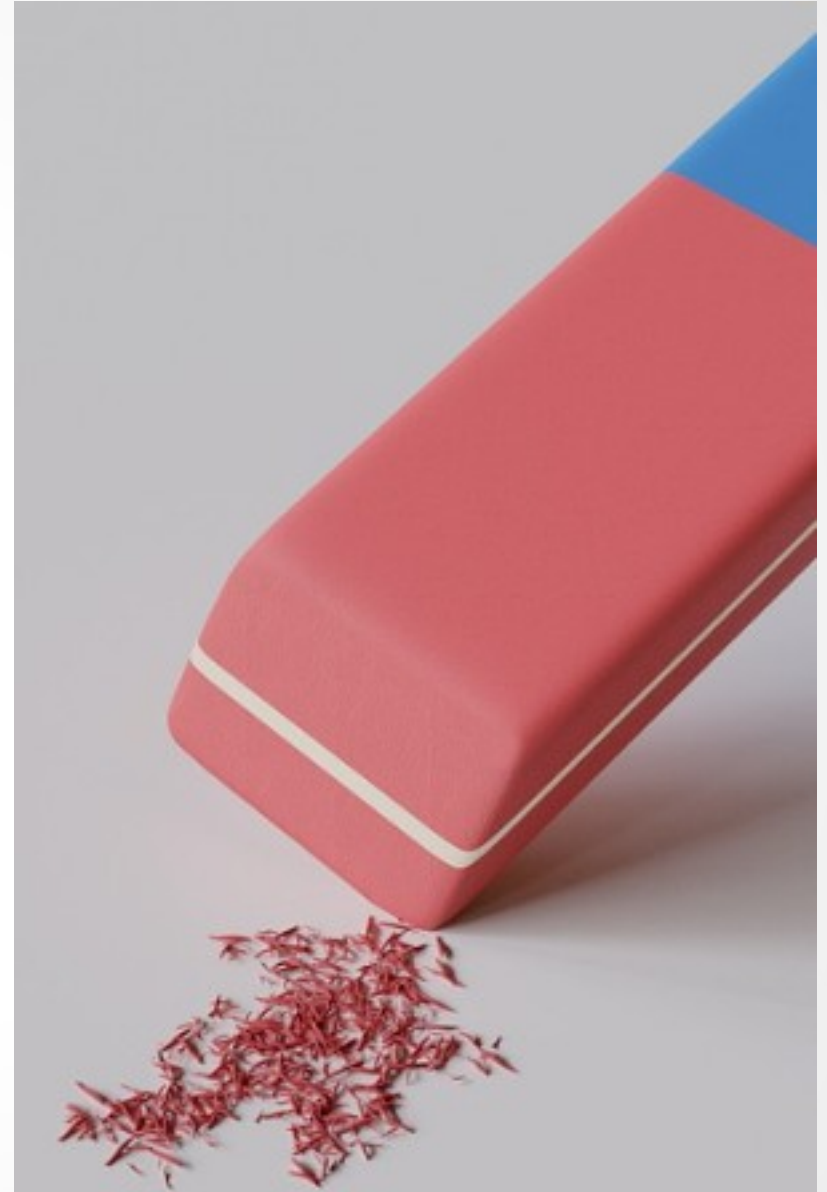
- Para eliminar una secuencia se utiliza el comando **DROP SEQUENCE**. Su sintaxis es:

DROP SEQUENCE nombre;

- Para eliminar una secuencia se debe ser el propietario de la secuencia o tener el privilegio **DROP ANY SEQUENCE** para eliminarla.

Secuencias presentes en el Sistema.

- **USER_SEQUENCES**. muestra las secuencias que tiene el usuario en su esquema.
- **ALL_SEQUENCES**. Muestra todas las secuencias presentes en la base de datos. Se diferencia de la anterior en que también muestra el propietario de la secuencia.



- A partir de **Oracle Database 12c** se puede prescindir de las secuencias empleando **columnas de identidad**. Una columna de identidad genera un valor numérico secuencial y único al estilo de otros gestores como MySQL y PostgreSQL.
- La sintaxis básica es la siguiente:

columna tipo GENERATED BY DEFAULT AS IDENTITY

- Veamos un ejemplo básico:

```
CREATE TABLE PROYECTOS (  
    PROYECTOID NUMBER(6) GENERATED BY DEFAULT AS IDENTITY,  
    NOMBRE VARCHAR2(60) NOT NULL,  
    CONSTRAINT PROYECTOS_PK PRIMARY KEY( PROJECT_ID)  
);
```

```
INSERT INTO PROYECTOS (PROJECT_NAME)  
VALUES ('PROYECTO ALFA');
```

```
INSERT INTO PROYECTOS (PROJECT_NAME)  
VALUES ('PROYECTO BETA');
```

```
SELECT * FROM PROYECTOS;
```

PROYECTOID	NOMBRE
-----	-----
1	ALFA
2	BETA

- Podemos añadir características de configuración a una columna IDENTITY tales como los valores mínimo y máximo, el incremento, etc. Para ello, usaremos la sintaxis siguiente:

columna tipo GENERATED BY DEFAULT AS IDENTITY

MINVALUE [mínimo]

MAXVALUE [máximo]

INCREMENT BY [incremento]

START WITH [inicio]

[NOORDER] [NOCYCLE] [NOT NULL] [ENABLE]

- En el ejemplo anterior podemos realizar la siguiente definición de la tabla.

```
CREATE TABLE PROYECTOS (  
    PROYECTOID NUMBER(6) GENERATED BY DEFAULT AS IDENTITY  
        MINVALUE 100  
        MAXVALUE 999999  
        INCREMENT BY 10  
        START WITH 100,  
    NOMBRE VARCHAR2(60) NOT NULL,  
    CONSTRAINT PROYECTOS_PK PRIMARY KEY( PROJECT_ID)  
);
```

- En este caso, los valores de PROYECTOID empiezan en el valor 100 y se incrementan de diez en diez.

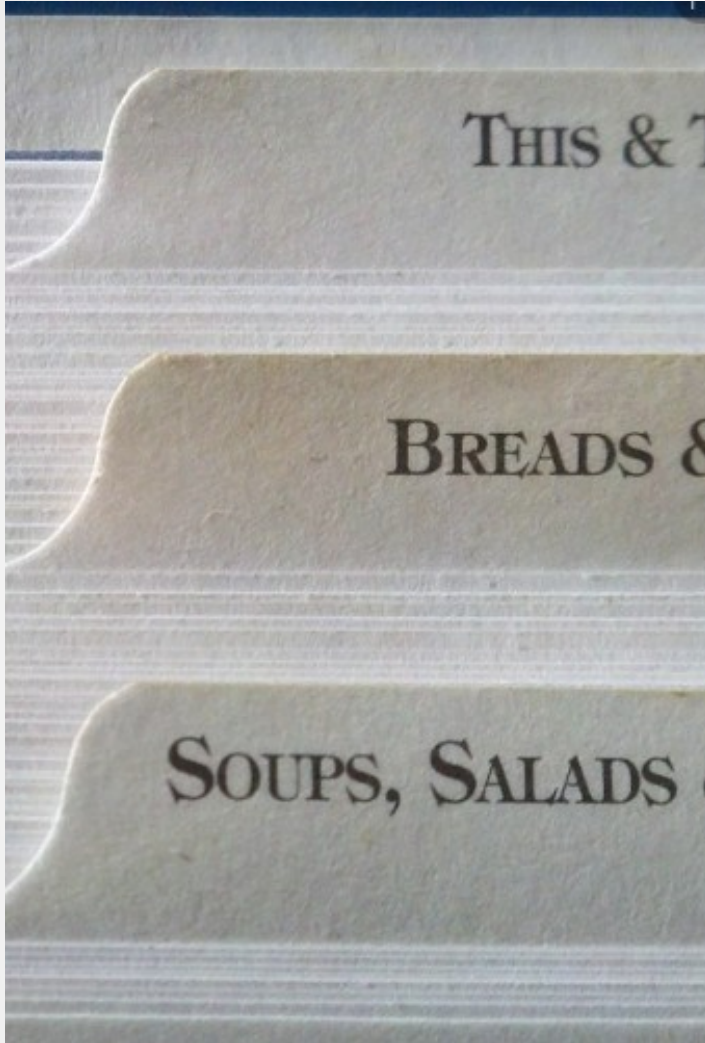
Definición.

- Mediante un índice el SGBD crea un puntero que es usado como método de acceso rápido a la localización de datos.
- La recuperación de filas se acelera de forma sensible ya que elimina el acceso a disco.
- El usuario no necesita realizar un proceso, la base de datos lo utiliza y mantiene automáticamente.
- Se pueden crear y borrar en cualquier momento y que no afectan a las tablas base o a otros índices. Sin embargo, al borrar una tabla, se borran también los índices correspondientes.

Tipos de Índices.

- **Índice único.** Se crea automáticamente un índice único cuando se define una restricción **PRIMARY KEY** o **UNIQUE**.
- **Índice no único.** Índice de columna **FOREIGN KEY** para una unión en una consulta para aumentar la velocidad de recuperación.





- La sintaxis del comando para crear un índice es la siguiente:

```
CREATE [UNIQUE] INDEX nombre  
ON tabla (columna1, columna2, ...)  
[COMPUTE STATISTICS];
```

- Cada uno de los parámetro tiene el siguiente significado:
 - **UNIQUE.** La combinación de valores en las columnas indexadas debe ser única.
 - **nombre.** Nombre del índice.
 - **tabla.** Nombre de la tabla.
 - **COMPUTE STATISTICS.** Oracle recolectará estadísticas durante la creación de la indexación. para que el optimizador escoga el mejor plan de ejecución cuando se ejecuten sentencias SQL.

- **Permiten denominar un objeto con otro nombre.**
- Puede crear sinónimos para poner un nombre alternativo a una tabla, vista, secuencias, procedimientos, funciones y otros objetos.
- Para hacer referencia a una tabla propiedad de otro usuario, debe poner como prefijo al nombre de tabla el nombre del usuario que la creó, seguido de un punto.
- Para que usuario pueda crear sinónimos tiene que tener el privilegio **CREATE SYNONYM**. Si no lo tiene mostrará un mensaje indicando que tiene privilegios insuficientes.
- Este método puede resultar especialmente útil con nombres de objeto largos, como las vistas. La sintaxis para la creación de un índice es la siguiente:

```
CREATE [OR REPLACE] [PUBLIC] SYNONYM [schema.] nombre  
FOR [schema.]object_name[@dblink];
```

- Los parámetros son los siguientes:
 - **OR REPLACE**. Crea nuevamente el sinónimo (si ya existe) sin tener que eliminarlo.
 - **schema**. Esquema que almacena el objeto. Si se omite, se asume el esquema actual.
 - **PUBLIC**. Accesible para todos los usuarios.
 - **@dblink**. Enlace con otra base de datos.

Eliminar un sinónimo.

- Para eliminar un sinónimo se utiliza la sentencia DROP SYNONYM. Su sintaxis es la siguiente:

DROP SYNONYM nombre;

Sinónimos presentes en el Sistema.

- **USER_SYNONYMS.** Muestra los sinónimos que tiene el usuario en su esquema.
- **ALL_SYNONYMS.** Muestra todos los sinónimos presentes en la base de datos. Se diferencia de la anterior en que también muestra el propietario del sinónimo.